

AI Ops Automation: Complete Technical Documentation

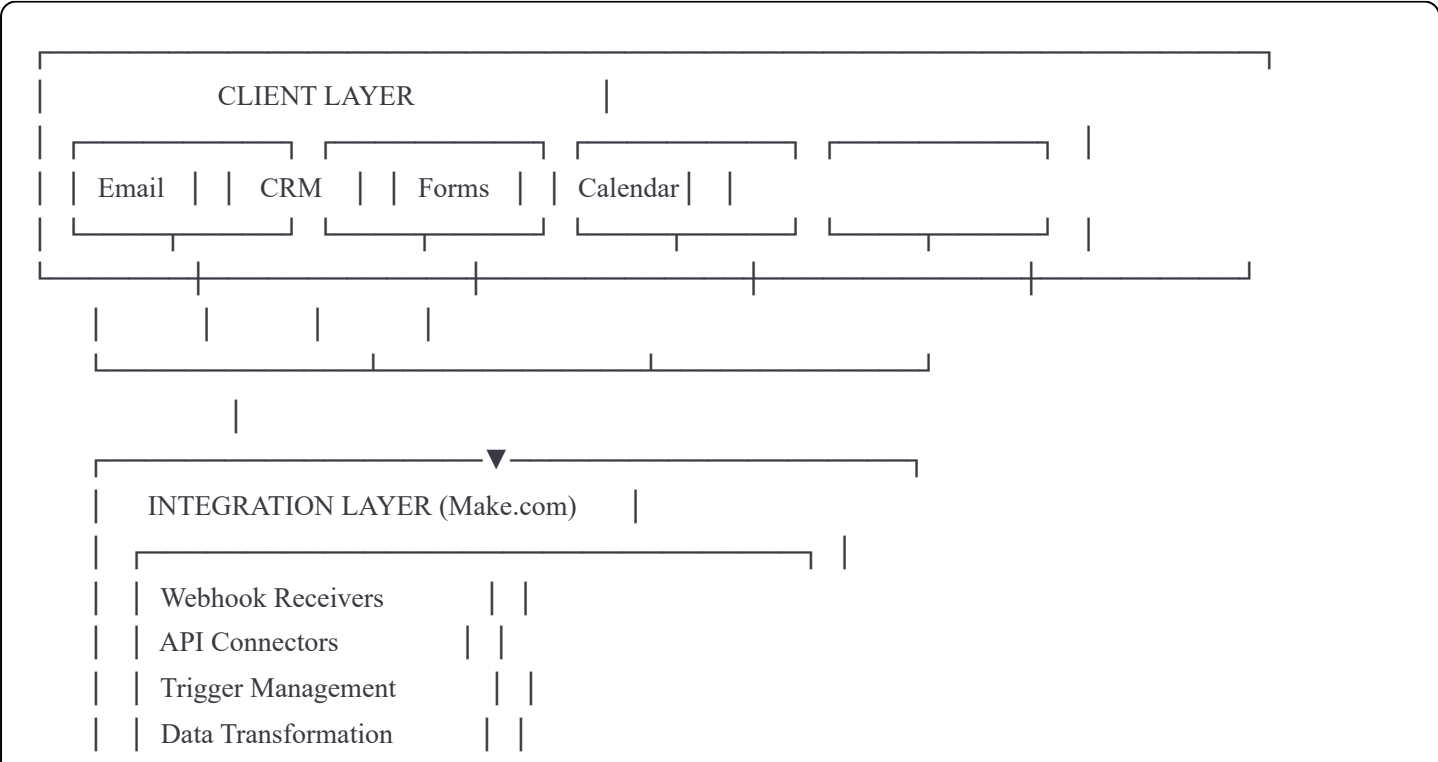
Architecture, Implementation & Code Examples

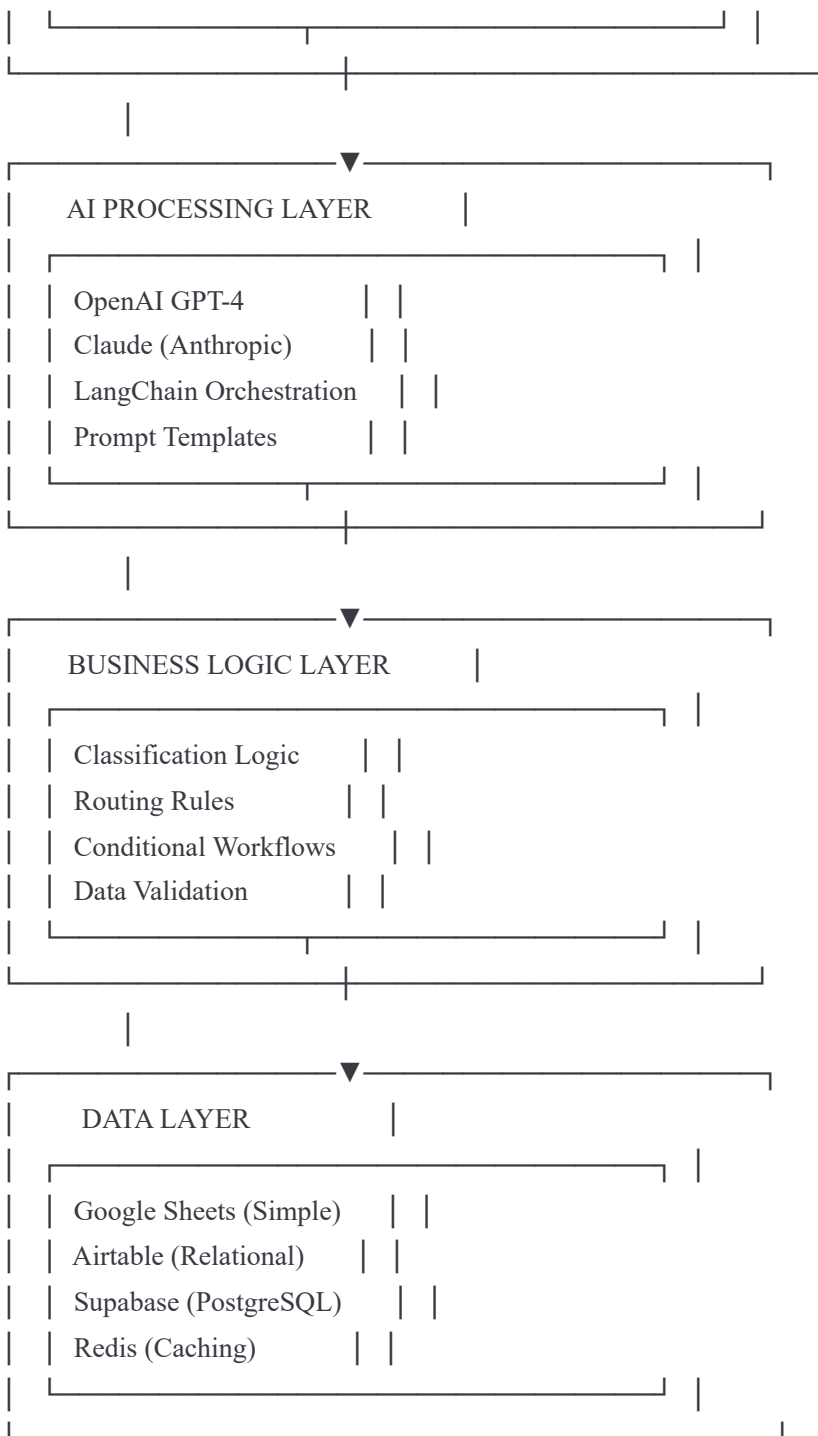
Table of Contents

- 1. [System Architecture Overview](#)
- 2. [Technology Stack Deep Dive](#)
- 3. [Core Automation Patterns](#)
- 4. [Implementation Guides](#)
- 5. [API Integration Examples](#)
- 6. [Database Design](#)
- 7. [Error Handling & Monitoring](#)
- 8. [Security & Authentication](#)
- 9. [Scaling Strategies](#)
- 10. [Code Library](#)

System Architecture Overview {#architecture}

High-Level Architecture





Component Responsibilities

Integration Layer (Make.com / n8n)

- **Purpose:** Connect different services without writing code
- **Handles:**
 - Webhook ingestion from external services
 - API calls to third-party platforms
 - Data transformation and mapping
 - Retry logic for failed operations
 - Rate limiting management

AI Processing Layer

- **Purpose:** Make intelligent decisions based on unstructured data
- **Handles:**
 - Natural language understanding
 - Content classification
 - Sentiment analysis
 - Response generation
 - Entity extraction

Business Logic Layer

- **Purpose:** Execute business rules and workflows
- **Handles:**
 - Decision trees based on AI output
 - Multi-step workflows
 - Conditional routing
 - Data validation
 - Business rule enforcement

Data Layer

- **Purpose:** Persist and retrieve data
- **Handles:**
 - User data storage
 - Conversation history
 - Analytics and logging
 - Configuration management
 - Cache management

Technology Stack Deep Dive {#tech-stack}

Phase 1: Zero Capital Start (Month 1-3)

No-Code Automation: Make.com

Why Make.com?

- Visual workflow builder (no coding required)
- 1,000 operations/month free tier
- 1,500+ app integrations
- Built-in error handling
- Webhook support

Key Concepts:

Scenario = Workflow

Module = Action/Trigger

Connection = Authenticated API link

Operation = Single execution (counted against quota)

Bundle = Set of data passed between modules

Make.com Structure:

javascript

Scenario: "Email Auto-Responder"

```
├─ Trigger: Gmail - Watch Emails
|   └─ Filters: Unread = true, Label = "Support"
|
├─ Module 1: OpenAI - Create Chat Completion
|   └─ Input: Email body
|   └─ Output: Classification (Sales/Support/Spam)
|
├─ Router (Split based on classification)
|   └─ Route 1: If "Sales"
|       └─ Gmail - Send Email (with sales template)
|       |
|       └─ Route 2: If "Support"
|           └─ Create Ticket in Airtable
|           └─ Gmail - Send Auto-reply
|           |
|           └─ Route 3: If "Spam"
|               └─ Gmail - Delete Email
|
└─ Final Module: Log to Google Sheets
    └─ Data: Timestamp, Classification, Response sent
```

Make.com Best Practices:

1. Error Handling:

- Add "Break" directive after critical modules
- Use "Ignore" for non-critical failures
- Set up error notification webhooks

2. Data Transformation:

- Use built-in functions: `{{formatDate()}}`, `{{trim()}}`, `{{lower()}}`
- Store complex transformations in separate modules
- Always validate data before API calls

3. Rate Limiting:

- Add "Sleep" modules between bulk operations
- Use Iterator for processing arrays (one at a time)
- Monitor operation count in dashboard

4. Testing:

- Use "Run Once" before activating
- Test with real data, not dummy data
- Check all conditional branches

AI Layer: OpenAI API

API Basics:

javascript

// Endpoint

POST <https://api.anthropic.com/v1/messages>

// Headers

```
{  
  "Content-Type": "application/json",  
  "x-api-key": "your-api-key",  
  "anthropic-version": "2023-06-01"  
}
```

// Request Body (for Claude)

```
{  
  "model": "claude-sonnet-4-20250514",  
  "max_tokens": 1024,  
  "messages": [  
    {  
      "role": "user",  
      "content": "Your prompt here"  
    }  
  ]  
}
```

// OpenAI Format (GPT-4)

```
{  
  "model": "gpt-4",  
  "messages": [  
    {  
      "role": "system",  
      "content": "You are a helpful assistant."  
    },  
    {  
      "role": "user",  
      "content": "User query here"  
    }  
  ],  
  "temperature": 0.7,  
  "max_tokens": 500  
}
```

Prompt Engineering Patterns:

javascript

// Pattern 1: Classification

```
const classificationPrompt = `
```

You are a classification system for customer emails.

Email Subject: \${subject}

Email Body: \${body}

Classify this email into EXACTLY ONE category:

- SALES: Customer wants to buy or inquire about products
- SUPPORT: Customer needs help with existing product/service
- BILLING: Questions about invoices, payments, refunds
- SPAM: Unsolicited or irrelevant content

Output ONLY the category name, nothing else.

```
`;
```

// Pattern 2: Extraction

```
const extractionPrompt = `
```

Extract structured information from this lead form submission:

\${formData}

Return a JSON object with these fields:

```
{  
  "name": "Full name",  
  "company": "Company name or null",  
  "email": "Email address",  
  "phone": "Phone number or null",  
  "budget": "Budget range or null",  
  "urgency": "ASAP | This Week | This Month | Flexible",  
  "needs": ["List", "of", "needs"]  
}
```

Return ONLY valid JSON, no explanation.

```
`;
```

// Pattern 3: Lead Scoring

```
const leadScoringPrompt = `
```

Analyze this lead and assign a score from 0-100:

Name: \${name}

Company: \${company}

Role: \${role}

Budget: \${budget}

Timeline: \${timeline}

Needs: \${needs}

Scoring criteria:

- Budget >₹50k: +30 points
- Timeline <1 month: +20 points
- Decision maker role: +20 points
- Clear specific needs: +15 points
- Company size >50: +15 points

Output format:

Score: [number]

Reasoning: [one sentence]

Action: [HOT | WARM | COLD]

Be strict. Most leads should be WARM or COLD.

`;

// Pattern 4: Response Generation

const responsePrompt = `

Generate a professional email response:

Context: Customer asked about \${topic}

Tone: \${businessTone} // Professional | Friendly | Urgent

Customer History: \${hasHistory ? "Existing customer" : "New inquiry"}

Guidelines:

- Keep under 150 words
- Include specific next steps
- **Add relevant link if applicable:** \${relevantLink}
- **Sign off as** \${businessName}

Customer Email:

\${customerEmail}

Generate response:

`;

Token Management:

javascript

```

// Estimate tokens (rough approximation)
function estimateTokens(text) {
  // 1 token ≈ 4 characters in English
  return Math.ceil(text.length / 4);
}

// Cost calculation (OpenAI GPT-4 pricing as of 2025)
const COST_PER_1K_INPUT = 0.03; // $0.03 per 1K input tokens
const COST_PER_1K_OUTPUT = 0.06; // $0.06 per 1K output tokens

function calculateCost(inputTokens, outputTokens) {
  const inputCost = (inputTokens / 1000) * COST_PER_1K_INPUT;
  const outputCost = (outputTokens / 1000) * COST_PER_1K_OUTPUT;
  return inputCost + outputCost;
}

// Example: Email classification
const emailBody = "500 word email...";
const inputTokens = estimateTokens(emailBody) + 100; // +100 for prompt
const outputTokens = 10; // Classification is short

const cost = calculateCost(inputTokens, outputTokens);
// Cost ≈ $0.002 per email classification

// Monthly cost for 1000 emails/day:
// 1000 emails * 30 days * $0.002 = $60/month

```

Data Storage: Google Sheets

Why Start with Google Sheets?

- Free, unlimited storage (within Google Drive limits)
- Built-in API access
- Familiar interface for clients
- Easy to visualize data
- Make.com has native integration

Sheet Structure Examples:

Sheet 1: "Leads"

Timestamp	Name	Email	Phone	Budget	Score	Status			
2025-01-15	John Doe	john@co.in	98xxxxx	₹75,000	85	HOT			
2025-01-15	Jane Sm..	jane@co.in	97xxxxx	₹25,000	45	WARM			

Sheet 2: "Email_Log"

Timestamp	From	Subject	Classification	Action					
2025-01-15	cust@mail.com	Need help	SUPPORT	Auto-reply					
2025-01-15	spam@bad.com	Buy now!	SPAM	Deleted					

Sheet 3: "Automation_Performance"

Date	Automation_Name	Runs	Success	Failures					
2025-01-15	Email Classifier	247	245	2					
2025-01-15	Lead Scorer	18	18	0					



Google Sheets API (via Make.com):

javascript

// Make.com Actions Available:

1. "Add a Row"

- Appends data to bottom of sheet
- Use for: Logging, new leads, records

2. "Update a Row"

- Modifies existing row by row number
- Use for: Status updates, scores

3. "Search Rows"

- Finds rows matching criteria
- Use for: Checking duplicates, lookups

4. "Get a Range"

- Reads multiple cells/rows
- Use for: Batch processing, reports

5. "Clear Values"

- Deletes content (keeps formatting)
- Use for: Resetting test data

// Example: Add Lead to Sheet

Module: Google Sheets - Add a Row

Spreadsheet: "Client Database"

Sheet: "Leads"

Values:

- Column A: {{timestamp}}
- Column B: {{name}}
- Column C: {{email}}
- Column D: {{phone}}
- Column E: {{budget}}
- Column F: {{ai_score}}
- Column G: {{status}}

Phase 2: Growing (Month 4-6)

Advanced Automation: n8n (Self-Hosted Alternative)

Why n8n?

- Self-hosted = unlimited operations (no quota)
- More control over data
- Custom JavaScript code nodes
- Better for complex logic

- One-time cost vs subscription

Installation (Basic Server Setup):

```
bash
```

Using Docker (Recommended)

```
docker run -it --rm \
  --name n8n \
  -p 5678:5678 \
  -v ~/.n8n:/home/node/.n8n \
  n8nio/n8n
```

Access at: http://localhost:5678

For production (with PostgreSQL):

```
docker-compose up -d
```

docker-compose.yml

```
version: '3.7'
```

```
services:
```

```
  postgres:
```

```
    image: postgres:14
```

```
    environment:
```

```
      POSTGRES_USER: n8n
```

```
      POSTGRES_PASSWORD: your_password
```

```
      POSTGRES_DB: n8n
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
```

```
  n8n:
```

```
    image: n8nio/n8n
```

```
    ports:
```

```
      - 5678:5678
```

```
    environment:
```

```
      - DB_TYPE=postgresdb
```

```
      - DB_POSTGRESDB_HOST=postgres
```

```
      - DB_POSTGRESDB_PORT=5432
```

```
      - DB_POSTGRESDB_DATABASE=n8n
```

```
      - DB_POSTGRESDB_USER=n8n
```

```
      - DB_POSTGRESDB_PASSWORD=your_password
```

```
      - N8N_BASIC_AUTH_ACTIVE=true
```

```
      - N8N_BASIC_AUTH_USER=admin
```

```
      - N8N_BASIC_AUTH_PASSWORD=your_password
```

```
    volumes:
```

```
      - n8n_data:/home/node/.n8n
```

```
    depends_on:
```

```
      - postgres
```

```
volumes:
```

postgres_data:
n8n_data:

n8n Advanced Features:

javascript

```
// 1. Code Node (JavaScript/Python)
```

```
// Access to full programming capabilities
```

```
// Example: Custom Lead Scoring Logic
```

```
const inputData = $input.all();
```

```
for (const item of inputData) {
```

```
  let score = 0;
```

```
  // Budget scoring
```

```
  const budget = parseInt(item.json.budget);
```

```
  if (budget > 100000) score += 40;
```

```
  else if (budget > 50000) score += 25;
```

```
  else if (budget > 20000) score += 10;
```

```
  // Timeline scoring
```

```
  const timeline = item.json.timeline.toLowerCase();
```

```
  if (timeline.includes('asap') || timeline.includes('urgent')) {
```

```
    score += 30;
```

```
  } else if (timeline.includes('week')) {
```

```
    score += 20;
```

```
  } else if (timeline.includes('month')) {
```

```
    score += 10;
```

```
  }
```

```
  // Company size scoring
```

```
  if (item.json.company_size > 100) score += 20;
```

```
  else if (item.json.company_size > 50) score += 10;
```

```
  // Decision maker scoring
```

```
  const role = item.json.role.toLowerCase();
```

```
  if (role.includes('ceo') || role.includes('founder') ||
```

```
    role.includes('director')) {
```

```
    score += 30;
```

```
  } else if (role.includes('manager')) {
```

```
    score += 15;
```

```
  }
```

```
  // Add score to output
```

```
  item.json.lead_score = score;
```

```
  // Classify
```

```
  if (score >= 70) item.json.classification = 'HOT';
```

```
  else if (score >= 40) item.json.classification = 'WARM';
```

```
  else item.json.classification = 'COLD';
```

```
}
```

```
return inputData;
```

```
// 2. Function Node
```

```
// Reusable functions across workflows
```

```
function calculateBusinessDays(startDate, endDate) {  
  let count = 0;  
  const current = new Date(startDate);  
  const end = new Date(endDate);  
  
  while (current <= end) {  
    const dayOfWeek = current.getDay();  
    if (dayOfWeek !== 0 && dayOfWeek !== 6) count++;  
    current.setDate(current.getDate() + 1);  
  }  
  
  return count;  
}
```

```
// 3. HTTP Request Node
```

```
// Custom API calls with full control
```

```
{  
  "method": "POST",  
  "url": "https://api.client-system.com/leads",  
  "authentication": "headerAuth",  
  "headerAuth": {  
    "name": "Authorization",  
    "value": "Bearer {{ $node['Get_Auth_Token'].json['access_token'] }}"  
  },  
  "body": {  
    "lead": {  
      "name": "{{ $json['name'] }}",  
      "email": "{{ $json['email'] }}",  
      "score": "{{ $json['lead_score'] }}",  
      "source": "AI_Automation"  
    }  
  },  
  "options": {  
    "timeout": 5000,  
    "retry": {  
      "maxTries": 3,  
      "waitBetweenTries": 1000  
    }  
  }  
}
```

```
}  
}
```

Database: Airtable

Why Airtable?

- Relational database capabilities
- Better than Google Sheets for complex data
- Built-in forms and views
- API access
- Affordable (\$20/month)

Base Structure Example:

```
javascript
```

// Base: "AI Automation CRM"

// Table 1: Clients

```
{
  "fields": {
    "Name": "Text",
    "Email": "Email",
    "Company": "Text",
    "Status": "Single Select", // Active, Paused, Churned
    "Package": "Single Select", // Starter, Growth, Enterprise
    "MRR": "Currency",
    "Start_Date": "Date",
    "Automations": "Link to Automations", // Many-to-Many
    "Contact_Log": "Link to Interactions" // One-to-Many
  }
}
```

// Table 2: Automations

```
{
  "fields": {
    "Name": "Text",
    "Type": "Single Select", // Email, CRM, Reporting, etc.
    "Status": "Single Select", // Active, Testing, Failed
    "Client": "Link to Clients", // Many-to-Many
    "Runs_Today": "Number",
    "Success_Rate": "Percent",
    "Last_Run": "Date",
    "Error_Count": "Number"
  }
}
```

// Table 3: Interactions

```
{
  "fields": {
    "Client": "Link to Clients",
    "Date": "Date",
    "Type": "Single Select", // Call, Email, Meeting
    "Notes": "Long Text",
    "Next_Action": "Text",
    "Attachments": "Attachment"
  }
}
```

// Table 4: Leads

```
{
  "fields": {
```

```
"Name": "Text",
"Email": "Email",
"Phone": "Phone",
"Company": "Text",
"Budget": "Currency",
"Score": "Number",
"Status": "Single Select", // New, Contacted, Qualified, Won, Lost
"Source": "Single Select", // LinkedIn, Website, Referral
"Notes": "Long Text",
"Assigned_To": "User" // If you have team
}
}
```

Airtable API Usage:

javascript

// Create Record

```
const AIRTABLE_API_KEY = "your_api_key";
const BASE_ID = "appXXXXXXXXXXXXXXXX";
const TABLE_NAME = "Leads";

const response = await fetch(
  `https://api.airtable.com/v0/${BASE_ID}/${TABLE_NAME}`,
  {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${AIRTABLE_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      "fields": {
        "Name": "John Doe",
        "Email": "john@example.com",
        "Score": 85,
        "Status": "New"
      }
    })
  }
);
```

```
const data = await response.json();
```

// Returns: { id: "recXXXXXXXXXXXXXXXX", fields: {...}, createTime: "..."}

// Update Record

```
await fetch(
  `https://api.airtable.com/v0/${BASE_ID}/${TABLE_NAME}/${recordId}`,
  {
    method: 'PATCH',
    headers: {
      'Authorization': `Bearer ${AIRTABLE_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      "fields": {
        "Status": "Contacted",
        "Notes": "Called and scheduled demo for Friday"
      }
    })
  }
);
```

// Query Records with Filter

```
const params = new URLSearchParams({
  filterByFormula: "AND({Score} > 70, {Status} = 'New')",
  sort: [JSON.stringify({field: "Score", direction: "desc"})],
  maxRecords: 10
});

const hotLeads = await fetch(
  `https://api.airtable.com/v0/${BASE_ID}/${TABLE_NAME}?${params}`,
  {
    headers: {
      'Authorization': `Bearer ${AIRTABLE_API_KEY}`
    }
  }
);
```

Phase 3: Scaling (Month 7-12)

Custom Backend: Node.js + PostgreSQL

When to Build Custom:

- You have 50+ automations running
- Need sub-second response times
- Handling sensitive data
- Want to white-label
- Make.com costs > ₹20k/month

Tech Stack:

Backend: Node.js + Express
Database: PostgreSQL (via Supabase)
Queue: BullMQ + Redis
AI: LangChain for orchestration
Hosting: Railway / Render / DigitalOcean

Basic Architecture:

javascript

// server.js - Main server file

```
const express = require('express');
const { createClient } = require('@supabase/supabase-js');
const OpenAI = require('openai');
const Queue = require('bullmq').Queue;
```

```
const app = express();
app.use(express.json());
```

// Initialize services

```
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_KEY
);

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY
});
```

```
const emailQueue = new Queue('email-processing', {
  connection: {
    host: process.env.REDIS_HOST,
    port: 6379
  }
});
```

// Webhook endpoint for incoming emails

```
app.post('/webhook/email', async (req, res) => {
  try {
    const { from, subject, body } = req.body;
```

// Add to queue for async processing

```
await emailQueue.add('classify-email', {
  from,
  subject,
  body,
  receivedAt: new Date()
});

res.status(200).json({ status: 'queued' });
} catch (error) {
  console.error(error);
  res.status(500).json({ error: 'Failed to queue email' });
}
});
```

```
// Queue processor
```

```
const { Worker } = require('bullmq');
```

```
new Worker('email-processing', async (job) => {
```

```
  const { from, subject, body } = job.data;
```

```
  // Step 1: Classify with AI
```

```
  const classification = await classifyEmail(subject, body);
```

```
  // Step 2: Store in database
```

```
  const { data, error } = await supabase
```

```
    .from('emails')
```

```
    .insert({
```

```
      from_address: from,
```

```
      subject,
```

```
      body,
```

```
      classification,
```

```
      processed_at: new Date()
```

```
    });
```

```
  // Step 3: Take action based on classification
```

```
  if (classification === 'HOT_LEAD') {
```

```
    await sendSMSAlert(from, subject);
```

```
    await createCRMRecord(from, subject, body);
```

```
  } else if (classification === 'SUPPORT') {
```

```
    await createSupportTicket(from, subject, body);
```

```
    await sendAutoReply(from);
```

```
  }
```

```
  return { success: true, classification };
```

```
});
```

```
async function classifyEmail(subject, body) {
```

```
  const response = await openai.chat.completions.create({
```

```
    model: "gpt-4",
```

```
    messages: [
```

```
      {
```

```
        role: "system",
```

```
        content: "Classify emails as: HOT_LEAD, WARM_LEAD, SUPPORT, SPAM"
```

```
      },
```

```
      {
```

```
        role: "user",
```

```
        content: `Subject: ${subject}\n\nBody: ${body}`
```

```
      }
```

```
    ],
```

```
    temperature: 0.3,
```

```
    max_tokens: 10
```

```
});

return response.choices[0].message.content.trim();
}

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Database Schema (PostgreSQL):

sql

-- Clients table

```
CREATE TABLE clients (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  company VARCHAR(255),  
  phone VARCHAR(50),  
  status VARCHAR(50) DEFAULT 'active',  
  package VARCHAR(50),  
  mrr DECIMAL(10, 2),  
  start_date DATE,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

-- Automations table

```
CREATE TABLE automations (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name VARCHAR(255) NOT NULL,  
  type VARCHAR(100),  
  status VARCHAR(50) DEFAULT 'active',  
  config JSONB, -- Store automation configuration  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Client automations (many-to-many)

```
CREATE TABLE client_automations (  
  client_id UUID REFERENCES clients(id) ON DELETE CASCADE,  
  automation_id UUID REFERENCES automations(id) ON DELETE CASCADE,  
  enabled BOOLEAN DEFAULT true,  
  PRIMARY KEY (client_id, automation_id)  
);
```

-- Automation runs (logging/analytics)

```
CREATE TABLE automation_runs (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  automation_id UUID REFERENCES automations(id),  
  client_id UUID REFERENCES clients(id),  
  status VARCHAR(50), -- success, failed, partial  
  input_data JSONB,  
  output_data JSONB,  
  error_message TEXT,  
  duration_ms INTEGER,  
  executed_at TIMESTAMP DEFAULT NOW()  
);
```

-- Leads table

```
CREATE TABLE leads (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name VARCHAR(255),  
  email VARCHAR(255),  
  phone VARCHAR(50),  
  company VARCHAR(255),  
  budget DECIMAL(10, 2),  
  score INTEGER,  
  status VARCHAR(50) DEFAULT 'new',  
  source VARCHAR(100),  
  notes TEXT,  
  assigned_to UUID REFERENCES clients(id),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Indexes for performance

```
CREATE INDEX idx_leads_status ON leads(status);  
CREATE INDEX idx_leads_score ON leads(score DESC);  
CREATE INDEX idx_automation_runs_executed ON automation_runs(executed_at DESC);  
CREATE INDEX idx_automation_runs_status ON automation_runs(status);
```

Core Automation Patterns {#patterns}

Pattern 1: Trigger-Action (Simple)

Use Case: Send reminder 1 day before appointment

TRIGGER: Daily at 9 AM

↓

ACTION: Check Google Calendar for tomorrow's appointments

↓

FILTER: Only appointments without "Confirmed" tag

↓

FOR EACH Appointment:

├─ Send SMS reminder

├─ Log in database

└─ Update appointment with "Reminder Sent" tag

Make.com Implementation:

javascript

1. **Schedule:** "Every day at 9:00 AM"
2. **Google Calendar:** "Search Events"
 - Calendar: "Appointments"
 - Start Time: {{now + 1 day, 00:00}}
 - End Time: {{now + 1 day, 23:59}}
3. **Filter:** Status != "Confirmed"
4. **Iterator:** Loop through events
5. **Twilio:** "Send SMS"
 - To: {{phone}}
 - Message: "Reminder: Appointment tomorrow at {{time}}"
6. **Google Calendar:** "Update Event"
 - Add note: "Reminder sent"
7. **Google Sheets:** "Add Row"
 - Log: Date, Name, Phone, Status

Pattern 2: AI Classification → Routing

Use Case: Triage incoming support tickets

TRIGGER: New email to support@company.com

↓

AI ANALYSIS:

- └─ Extract: Issue type, urgency, customer sentiment
- └─ Classify severity: CRITICAL | HIGH | MEDIUM | LOW
- └─ Generate suggested response

↓

ROUTER (based on severity):

- └─ CRITICAL → SMS to team + Create urgent ticket + Auto-reply
- └─ HIGH → Email to team + Create ticket + Auto-reply
- └─ MEDIUM → Create ticket + Auto-reply
- └─ LOW → Check FAQ, if found auto-reply, else create ticket

Implementation with Code:

javascript

// AI Classification Function

```
async function classifySupport(email) {
```

```
  const prompt = `
```

Analyze this support email and return JSON:

```
Subject: ${email.subject}
```

```
Body: ${email.body}
```

```
Customer: ${email.from}
```

Return:

```
{
  "severity": "CRITICAL|HIGH|MEDIUM|LOW",
  "category": "Bug|Feature|Question|Billing",
  "sentiment": "Angry|Frustrated|Neutral|Happy",
  "urgency_score": 0-100,
  "summary": "One sentence summary",
  "suggested_response": "Draft response"
}
```

Severity rules:

- CRITICAL: Service down, data loss, security issue
- HIGH: Major feature broken, revenue impacting
- MEDIUM: Minor bugs, one user affected
- LOW: Questions, feature requests

```
`;
```

```
const response = await openai.chat.completions.create({
  model: "gpt-4",
  messages: [{ role: "user", content: prompt }],
  response_format: { type: "json_object" },
  temperature: 0.2
});
```

```
return JSON.parse(response.choices[0].message.content);
}
```

// Routing Logic

```
async function handleSupportEmail(email) {
```

```
  const analysis = await classifySupport(email);
```

// Store in database

```
const ticket = await supabase.from('tickets').insert({
  email_from: email.from,
  subject: email.subject,
  body: email.body,
  severity: analysis.severity,
```

```
category: analysis.category,
sentiment: analysis.sentiment,
urgency_score: analysis.urgency_score,
status: 'new'
}).select().single();

// Route based on severity
switch(analysis.severity) {
case 'CRITICAL':
  await sendSMS(Team_Phone, `CRITICAL: ${analysis.summary}`);
  await sendEmail(Team_Email, {
    subject: `🔴 CRITICAL: ${email.subject}`,
    body: `${analysis.summary}\n\nTicket: ${ticket.id}`
  });
  await sendAutoReply(email.from, {
    template: 'critical',
    variables: { ticket_id: ticket.id }
  });
  break;

case 'HIGH':
  await sendEmail(Team_Email, {
    subject: `⚠️ HIGH: ${email.subject}`,
    body: `${analysis.summary}\n\nTicket: ${ticket.id}`
  });
  await sendAutoReply(email.from, {
    template: 'high',
    variables: {
      ticket_id: ticket.id,
      eta: '4 hours'
    }
  });
  break;

case 'MEDIUM':
  await sendAutoReply(email.from, {
    template: 'medium',
    variables: {
      ticket_id: ticket.id,
      eta: '24 hours'
    }
  });
  break;

case 'LOW':
  // Check FAQ first
  const faqAnswer = await searchFAQ(analysis.summary);
```

```

if (faqAnswer) {
  await sendAutoReply(email.from, {
    template: 'faq',
    variables: { answer: faqAnswer }
  });
  // Mark ticket as auto-resolved
  await supabase.from('tickets')
    .update({ status: 'auto_resolved' })
    .eq('id', ticket.id);
} else {
  await sendAutoReply(email.from, {
    template: 'low',
    variables: { ticket_id: ticket.id }
  });
}
break;
}

return ticket;
}

```

Pattern 3: Scheduled Aggregation + Reporting

Use Case: Daily performance report

TRIGGER: Every day at 8 AM

↓

DATA COLLECTION:

- └─ Query database for yesterday's metrics
- └─ Calculate KPIs
- └─ Compare to previous day/week

↓

AI ANALYSIS:

- └─ Identify trends
- └─ Flag anomalies
- └─ Generate insights

↓

REPORT GENERATION:

- └─ Create formatted email/PDF
- └─ Include charts/visualizations
- └─ Add AI commentary

↓

DISTRIBUTION:

- └─ Email to stakeholders
- └─ Post to Slack channel

Implementation:

javascript

// Daily Report Generator

```
async function generateDailyReport() {  
  // 1. Collect data  
  const yesterday = new Date();  
  yesterday.setDate(yesterday.getDate() - 1);  
  
  const metrics = await collectMetrics(yesterday);  
  
  // 2. AI Analysis  
  const insights = await analyzeMetrics(metrics);  
  
  // 3. Generate report  
  const report = await createReport(metrics, insights);  
  
  // 4. Distribute  
  await emailReport(report);  
  await postToSlack(report);  
}
```

```
async function collectMetrics(date) {  
  const dateStr = date.toISOString().split('T')[0];
```

// Query automation runs

```
const { data: runs } = await supabase  
  .from('automation_runs')  
  .select('status, duration_ms, automation_id')  
  .gte('executed_at', `${dateStr}T00:00:00`)  
  .lt('executed_at', `${dateStr}T23:59:59`);
```

// Calculate metrics

```
const totalRuns = runs.length;  
const successRuns = runs.filter(r => r.status === 'success').length;  
const failedRuns = runs.filter(r => r.status === 'failed').length;  
const successRate = (successRuns / totalRuns * 100).toFixed(2);  
const avgDuration = runs.reduce((sum, r) => sum + r.duration_ms, 0) / totalRuns;
```

// Query leads

```
const { data: leads } = await supabase  
  .from('leads')  
  .select('score, status')  
  .gte('created_at', `${dateStr}T00:00:00`)  
  .lt('created_at', `${dateStr}T23:59:59`);
```

```
const newLeads = leads.length;  
const hotLeads = leads.filter(l => l.score >= 70).length;  
const avgScore = leads.reduce((sum, l) => sum + l.score, 0) / newLeads;
```

```
return {
  date: dateStr,
  automations: {
    totalRuns,
    successRuns,
    failedRuns,
    successRate,
    avgDuration
  },
  leads: {
    newLeads,
    hotLeads,
    avgScore
  }
};
}
```

```
async function analyzeMetrics(metrics) {
  const prompt = `
Analyze this daily performance data and provide insights:
```

```
${JSON.stringify(metrics, null, 2)}
```

Provide:

1. Key highlights (2-3 bullet points)
2. Concerns or anomalies (if any)
3. Actionable recommendations (1-2)

Keep it concise and business-focused.

```
`;
```

```
const response = await openai.chat.completions.create({
  model: "gpt-4",
  messages: [{ role: "user", content: prompt }],
  temperature: 0.7,
  max_tokens: 300
});

return response.choices[0].message.content;
}
```

```
async function createReport(metrics, insights) {
  const html = `
<!DOCTYPE html>
<html>
<head>
```

```
<style>
  body { font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; }
  .metric { background: #f5f5f5; padding: 15px; margin: 10px 0; border-radius: 5px; }
  .metric h3 { margin: 0 0 10px 0; color: #333; }
  .metric p { margin: 5px 0; font-size: 24px; font-weight: bold; }
  .good { color: #4CAF50; }
  .warning { color: #FF9800; }
  .bad { color: #F44336; }
  .insights { background: #e3f2fd; padding: 15px; border-left: 4px solid #2196F3; }
</style>
</head>
<body>
  <h1>Daily Performance Report</h1>
  <p><strong>Date:</strong> ${metrics.date}</p>

  <h2>Automations</h2>
  <div class="metric">
    <h3>Total Runs</h3>
    <p>${metrics.automations.totalRuns}</p>
  </div>

  <div class="metric">
    <h3>Success Rate</h3>
    <p class="${metrics.automations.successRate >= 95 ? 'good' : metrics.automations.successRate >= 90 ? 'warning' : 'bad'}">
      ${metrics.automations.successRate}%
    </p>
    <small>${metrics.automations.successRuns} succeeded, ${metrics.automations.failedRuns} failed</small>
  </div>

  <div class="metric">
    <h3>Avg Response Time</h3>
    <p>${(metrics.automations.avgDuration / 1000).toFixed(2)}s</p>
  </div>

  <h2>Leads</h2>
  <div class="metric">
    <h3>New Leads</h3>
    <p>${metrics.leads.newLeads}</p>
  </div>

  <div class="metric">
    <h3>Hot Leads</h3>
    <p class="good">${metrics.leads.hotLeads}</p>
    <small>${((metrics.leads.hotLeads / metrics.leads.newLeads) * 100).toFixed(1)}% of total</small>
  </div>

  <div class="metric">
```

```

    <h3>Avg Lead Score</h3>
    <p>${metrics.leads.avgScore.toFixed(1)}</p>
  </div>

  <h2>AI Insights</h2>
  <div class="insights">
    ${insights.split('\n').map(line => `<p>${line}</p>`).join("")}
  </div>
</body>
</html>
`;

  return html;
}

async function emailReport(html) {
  // Using SendGrid, AWS SES, or similar
  await sendEmail({
    to: 'team@company.com',
    subject: `Daily Report - ${new Date().toLocaleDateString}`,
    html: html
  });
}

```

Pattern 4: Event-Driven Workflow

Use Case: Multi-step lead nurturing

TRIGGER: New lead form submission



IMMEDIATE:

- └─ Score lead with AI
- └─ Store in database
- └─ Send confirmation email to lead
- └─ Notify sales team if hot



DELAYED (2 hours):

- └─ Check if lead opened email
- └─ If yes: Send resource link
- └─ If no: Continue



DELAYED (1 day):

- └─ Check if lead clicked link
- └─ If yes: Send calendar booking link
- └─ If no: Send reminder email



DELAYED (3 days):

- └─ Check if lead booked call
- └─ If yes: Stop sequence
- └─ If no: Send final follow-up

Implementation with BullMQ:

javascript

```
const { Queue, Worker } = require('bullmq');
const Redis = require('ioredis');

const connection = new Redis({
  host: 'localhost',
  port: 6379,
  maxRetriesPerRequest: null
});

// Create queue
const leadNurturingQueue = new Queue('lead-nurturing', { connection });

// When new lead comes in
async function handleNewLead(leadData) {
  // Immediate: Score and store
  const score = await scoreLead(leadData);
  const lead = await supabase.from('leads').insert({
    ...leadData,
    score,
    status: 'new'
  }).select().single();

  // Immediate: Send confirmation
  await sendEmail({
    to: lead.email,
    template: 'lead_confirmation',
    variables: { name: lead.name }
  });

  // If hot lead, notify immediately
  if (score >= 70) {
    await sendSMS(SALES_TEAM, `Hot lead: ${lead.name} (${score})`);
  }

  // Schedule follow-up sequence
  await leadNurturingQueue.add(
    'check-email-open',
    { leadId: lead.id },
    { delay: 2 * 60 * 60 * 1000 } // 2 hours
  );

  await leadNurturingQueue.add(
    'send-day-1-followup',
    { leadId: lead.id },
    { delay: 24 * 60 * 60 * 1000 } // 1 day
  );
}
```

```
await leadNurturingQueue.add(
  'send-day-3-followup',
  { leadId: lead.id },
  { delay: 3 * 24 * 60 * 60 * 1000 } // 3 days
);
}

// Worker to process jobs
new Worker('lead-nurturing', async (job) => {
  const { leadId } = job.data;

  // Get latest lead data
  const { data: lead } = await supabase
    .from('leads')
    .select('*')
    .eq('id', leadId)
    .single();

  // If lead already converted, stop
  if (lead.status === 'converted') {
    return { skipped: true, reason: 'Already converted' };
  }

  switch(job.name) {
    case 'check-email-open':
      const opened = await checkEmailOpen(lead.email);
      if (opened) {
        await sendEmail({
          to: lead.email,
          template: 'resource_link',
          variables: { name: lead.name }
        });
        await supabase.from('leads')
          .update({ email_opened: true })
          .eq('id', leadId);
      }
      break;

    case 'send-day-1-followup':
      const clickedLink = await checkLinkClick(lead.email);
      if (clickedLink) {
        await sendEmail({
          to: lead.email,
          template: 'booking_link',
          variables: {
            name: lead.name,
```

```

        calendly_link: CALENDLY_URL
    }
});
} else {
    await sendEmail({
        to: lead.email,
        template: 'reminder',
        variables: { name: lead.name }
    });
}
break;

case 'send-day-3-followup':
    const booked = await checkIfBooked(lead.email);
    if (!booked) {
        await sendEmail({
            to: lead.email,
            template: 'final_followup',
            variables: { name: lead.name }
        });
        await supabase.from('leads')
            .update({ status: 'cold' })
            .eq('id', leadId);
    }
    break;
}
}, { connection });

```

Pattern 5: Data Enrichment Pipeline

Use Case: Enhance lead data automatically

TRIGGER: New lead with minimal info (just email)



STEP 1: Email Verification

- └─ Check if email is valid
- └─ If invalid, mark and alert



STEP 2: Company Lookup

- └─ Extract domain from email
- └─ Look up company via Clearbit/Hunter.io
- └─ Store: Company name, size, industry



STEP 3: Person Lookup

- └─ Search LinkedIn Sales Navigator
- └─ Store: Job title, seniority



STEP 4: Scoring

- └─ Score based on enriched data
- └─ Update lead classification



STEP 5: Assignment

- └─ Route to appropriate sales rep
- └─ Add to nurture campaign

Implementation:

javascript

```
async function enrichLead(email) {  
  const pipeline = [];  
  
  // Step 1: Validate email  
  pipeline.push(async () => {  
    const isValid = await validateEmail(email);  
    if (!isValid) throw new Error('Invalid email');  
    return { email, valid: true };  
  });  
  
  // Step 2: Get company data  
  pipeline.push(async (data) => {  
    const domain = email.split('@')[1];  
    const company = await lookupCompany(domain);  
    return { ...data, company };  
  });  
  
  // Step 3: Get person data  
  pipeline.push(async (data) => {  
    const person = await lookupPerson(email);  
    return { ...data, person };  
  });  
  
  // Step 4: AI Scoring  
  pipeline.push(async (data) => {  
    const score = await calculateScore(data);  
    return { ...data, score };  
  });  
  
  // Execute pipeline  
  let result = { email };  
  for (const step of pipeline) {  
    try {  
      result = await step(result);  
    } catch (error) {  
      console.error(`Pipeline step failed: ${error.message}`);  
      result.errors = result.errors || [];  
      result.errors.push(error.message);  
    }  
  }  
  
  // Store enriched lead  
  await supabase.from('leads').insert(result);  
  
  return result;  
}
```

```
async function validateEmail(email) {  
  // Use service like ZeroBounce or Email Verification API  
  const response = await fetch(  
    `https://api.zerobounce.net/v2/validate?api_key=${API_KEY}&email=${email}`  
  );  
  const data = await response.json();  
  return data.status === 'valid';  
}
```

```
async function lookupCompany(domain) {  
  // Use Clearbit Company API  
  try {  
    const response = await fetch(  
      `https://company.clearbit.com/v2/companies/find?domain=${domain}`,  
      {  
        headers: { 'Authorization': `Bearer ${CLEARBIT_KEY}` }  
      }  
    );  
    const data = await response.json();  
    return {  
      name: data.name,  
      domain: data.domain,  
      industry: data.category.industry,  
      employees: data.metrics.employees,  
      revenue: data.metrics.estimatedAnnualRevenue  
    };  
  } catch (error) {  
    return null;  
  }  
}
```

```
async function calculateScore(data) {  
  let score = 0;  
  
  // Company size scoring  
  if (data.company?.employees > 500) score += 30;  
  else if (data.company?.employees > 100) score += 20;  
  else if (data.company?.employees > 50) score += 10;  
  
  // Revenue scoring  
  if (data.company?.revenue > 10000000) score += 25;  
  else if (data.company?.revenue > 5000000) score += 15;  
  
  // Seniority scoring  
  const title = data.person?.title?.toLowerCase() || '';  
  if (title.includes('ceo') || title.includes('founder')) score += 35;
```

```
else if (title.includes('director') || title.includes('vp')) score += 25;
else if (title.includes('manager')) score += 15;

// Industry fit (customize based on your niche)
const targetIndustries = ['Technology', 'SaaS', 'E-commerce'];
if (targetIndustries.includes(data.company?.industry)) score += 15;

return Math.min(score, 100);
}
```

API Integration Examples {#api-integration}

Gmail API

Setup:

1. Go to Google Cloud Console
2. Enable Gmail API
3. Create OAuth 2.0 credentials
4. Get refresh token

Reading Emails:

```
javascript
```

```
const { google } = require('googleapis');

const oauth2Client = new google.auth.OAuth2(
  CLIENT_ID,
  CLIENT_SECRET,
  REDIRECT_URI
);

oauth2Client.setCredentials({
  refresh_token: REFRESH_TOKEN
});

const gmail = google.gmail({ version: 'v1', auth: oauth2Client });

async function getUnreadEmails() {
  const response = await gmail.users.messages.list({
    userId: 'me',
    q: 'is:unread label:inbox',
    maxResults: 10
  });

  const messages = response.data.messages || [];

  const emails = await Promise.all(
    messages.map(async (msg) => {
      const details = await gmail.users.messages.get({
        userId: 'me',
        id: msg.id,
        format: 'full'
      });

      const headers = details.data.payload.headers;
      const subject = headers.find(h => h.name === 'Subject')?.value;
      const from = headers.find(h => h.name === 'From')?.value;

      // Decode body
      let body = "";
      if (details.data.payload.body.data) {
        body = Buffer.from(
          details.data.payload.body.data,
          'base64'
        ).toString('utf-8');
      }

      return { id: msg.id, from, subject, body };
    })
  );
}
```

```
);

return emails;
}

async function sendEmail(to, subject, body) {
  const raw = [
    `To: ${to}`,
    `Subject: ${subject}`,
    "",
    body
  ].join("\n");

  const encodedMessage = Buffer.from(raw)
    .toString('base64')
    .replace(/\+/g, '-')
    .replace(/\//g, '_')
    .replace(/=+$/, "");

  await gmail.users.messages.send({
    userId: 'me',
    requestBody: {
      raw: encodedMessage
    }
  });
}
```

Twilio (SMS/WhatsApp)

javascript

```

const twilio = require('twilio');

const client = twilio(ACCOUNT_SID, AUTH_TOKEN);

// Send SMS
async function sendSMS(to, message) {
  const result = await client.messages.create({
    body: message,
    to: to, // Format: +91XXXXXXXXXX
    from: TWILIO_PHONE_NUMBER
  });

  return result.sid;
}

// Send WhatsApp
async function sendWhatsApp(to, message) {
  const result = await client.messages.create({
    body: message,
    to: `whatsapp:${to}`, // Format: whatsapp:+91XXXXXXXXXX
    from: `whatsapp:${TWILIO_WHATSAPP_NUMBER}`
  });

  return result.sid;
}

// Receive SMS webhook handler
app.post('/webhook/sms', (req, res) => {
  const { From, Body } = req.body;

  console.log(`SMS from ${From}: ${Body}`);

  // Process the SMS (e.g., update appointment confirmation)
  processSMSReply(From, Body);

  // Respond with TwiML if you want to reply
  const twiml = new twilio.twiml.MessagingResponse();
  twiml.message('Thank you! Your response has been recorded.');
```

```

  res.type('text/xml');
  res.send(twiml.toString());
});

```

HubSpot CRM

javascript

```
const hubspot = require('@hubspot/api-client');
```

```
const hubspotClient = new hubspot.Client({  
  accessToken: HUBSPOT_ACCESS_TOKEN  
});
```

```
// Create contact
```

```
async function createContact(email, firstName, lastName, company) {  
  const contact = await hubspotClient.crm.contacts.basicApi.create({  
    properties: {  
      email,  
      firstname: firstName,  
      lastname: lastName,  
      company  
    }  
  });  
  
  return contact.id;  
}
```

```
// Create deal
```

```
async function createDeal(contactId, dealName, amount, stage) {  
  const deal = await hubspotClient.crm.deals.basicApi.create({  
    properties: {  
      dealname: dealName,  
      amount: amount,  
      dealstage: stage,  
      pipeline: 'default'  
    },  
    associations: [  
      {  
        to: { id: contactId },  
        types: [  
          {  
            associationCategory: 'HUBSPOT_DEFINED',  
            associationTypeId: 3 // Contact to Deal  
          }  
        ]  
      }  
    ]  
  });  
  
  return deal.id;  
}
```

```
// Search contacts
```

```
async function searchContacts(email) {  
  const result = await hubspotClient.crm.contacts.searchApi.doSearch({  
    filterGroups: [  
      {  
        filters: [  
          {  
            propertyName: 'email',  
            operator: 'EQ',  
            value: email  
          }  
        ]  
      }  
    ],  
    properties: ['email', 'firstname', 'lastname', 'company'],  
    limit: 1  
  });  
  
  return result.results[0] || null;  
}
```

Slack Integration

javascript

```
const { WebClient } = require('@slack/web-api');
```

```
const slack = new WebClient(SLACK_BOT_TOKEN);
```

```
// Post message to channel
```

```
async function postToSlack(channel, message) {  
  await slack.chat.postMessage({  
    channel: channel, // #general or channel ID  
    text: message,  
    blocks: [  
      {  
        type: 'section',  
        text: {  
          type: 'mrkdwn',  
          text: message  
        }  
      }  
    ]  
  });  
}
```

```
// Post rich message with buttons
```

```
async function postInteractiveMessage(channel, leadData) {  
  await slack.chat.postMessage({  
    channel: channel,  
    text: `New hot lead: ${leadData.name}`,  
    blocks: [  
      {  
        type: 'header',  
        text: {  
          type: 'plain_text',  
          text: '🔥 New Hot Lead!'  
        }  
      },  
      {  
        type: 'section',  
        fields: [  
          {  
            type: 'mrkdwn',  
            text: `*Name:* \n${leadData.name}`  
          },  
          {  
            type: 'mrkdwn',  
            text: `*Company:* \n${leadData.company}`  
          }  
        ]  
      }  
    ]  
  });  
}
```

```

      type: 'mrkdwn',
      text: `*Score:*\\n${leadData.score}/100`
    },
    {
      type: 'mrkdwn',
      text: `*Budget:*\\n₹${leadData.budget}`
    }
  ]
},
{
  type: 'actions',
  elements: [
    {
      type: 'button',
      text: {
        type: 'plain_text',
        text: 'Claim Lead'
      },
      style: 'primary',
      value: leadData.id,
      action_id: 'claim_lead'
    },
    {
      type: 'button',
      text: {
        type: 'plain_text',
        text: 'View in CRM'
      },
      url: `https://crm.company.com/leads/${leadData.id}`
    }
  ]
}
});
}

```

// Handle button clicks (requires separate endpoint)

```

app.post('/slack/interactions', async (req, res) => {
  const payload = JSON.parse(req.body.payload);

  if (payload.actions[0].action_id === 'claim_lead') {
    const leadId = payload.actions[0].value;
    const userId = payload.user.id;

    // Assign lead
    await assignLead(leadId, userId);
  }
}

```

```
// Update message
```

```
await slack.chat.update({  
  channel: payload.channel.id,  
  ts: payload.message.ts,  
  text: `Lead claimed by <@${userId}>`,  
  blocks: payload.message.blocks  
});  
}  
  
res.sendStatus(200);  
});
```

Error Handling & Monitoring {#error-handling}

Error Handling Patterns

```
javascript
```

// Pattern 1: Retry with Exponential Backoff

```
async function retryWithBackoff(fn, maxRetries = 3) {  
  for (let i = 0; i < maxRetries; i++) {  
    try {  
      return await fn();  
    } catch (error) {  
      if (i === maxRetries - 1) throw error;  
  
      const delay = Math.pow(2, i) * 1000; // 1s, 2s, 4s  
      console.log(`Retry ${i + 1}/${maxRetries} after ${delay}ms`);  
      await new Promise(resolve => setTimeout(resolve, delay));  
    }  
  }  
}
```

// Usage

```
const result = await retryWithBackoff(async () => {  
  return await callExternalAPI();  
});
```

// Pattern 2: Circuit Breaker

```
class CircuitBreaker {  
  constructor(threshold = 5, timeout = 60000) {  
    this.failureCount = 0;  
    this.threshold = threshold;  
    this.timeout = timeout;  
    this.state = 'CLOSED'; // CLOSED, OPEN, HALF_OPEN  
    this.nextAttempt = Date.now();  
  }  
  
  async execute(fn) {  
    if (this.state === 'OPEN') {  
      if (Date.now() < this.nextAttempt) {  
        throw new Error('Circuit breaker is OPEN');  
      }  
      this.state = 'HALF_OPEN';  
    }  
  
    try {  
      const result = await fn();  
      this.onSuccess();  
      return result;  
    } catch (error) {  
      this.onFailure();  
      throw error;  
    }  
  }  
}
```

```

    }

    onSuccess() {
      this.failureCount = 0;
      this.state = 'CLOSED';
    }

    onFailure() {
      this.failureCount++;
      if (this.failureCount >= this.threshold) {
        this.state = 'OPEN';
        this.nextAttempt = Date.now() + this.timeout;
        console.log(`Circuit breaker opened for ${this.timeout}ms`);
      }
    }
  }
}

```

// Usage

```

const breaker = new CircuitBreaker();
const result = await breaker.execute(async () => {
  return await unreliableAPI();
});

```

// Pattern 3: Graceful Degradation

```

async function getLeadScore(leadData) {
  try {
    // Try AI scoring first
    return await aiScore(leadData);
  } catch (error) {
    console.warn('AI scoring failed, using fallback:', error);

    // Fall back to rule-based scoring
    return ruleBasedScore(leadData);
  }
}

```

```

function ruleBasedScore(leadData) {
  let score = 0;
  if (leadData.budget > 50000) score += 30;
  if (leadData.timeline === 'urgent') score += 30;
  if (leadData.company_size > 50) score += 20;
  return score;
}

```

Logging & Monitoring

javascript

// Structured logging with Winston

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});
```

// Log automation execution

```
async function executeAutomation(automation, data) {
  const startTime = Date.now();
  const logContext = {
    automationId: automation.id,
    automationName: automation.name,
    inputData: data
  };
}
```

```
logger.info('Automation started', logContext);
```

```
try {
  const result = await automation.execute(data);

  const duration = Date.now() - startTime;
  logger.info('Automation completed', {
    ...logContext,
    duration,
    status: 'success',
    outputData: result
  });
}
```

// Store metrics

```
await storeMetrics({
  automation_id: automation.id,
  status: 'success',
  duration,
  executed_at: new Date()
});
```

```
    return result;
  } catch (error) {
    const duration = Date.now() - startTime;
    logger.error('Automation failed', {
      ...logContext,
      duration,
      status: 'failed',
      error: error.message,
      stack: error.stack
    });

    // Store failure
    await storeMetrics({
      automation_id: automation.id,
      status: 'failed',
      duration,
      error_message: error.message,
      executed_at: new Date()
    });

    // Alert if critical
    if (automation.critical) {
      await sendAlert({
        type: 'automation_failure',
        automation: automation.name,
        error: error.message
      });
    }

    throw error;
  }
}
```

Health Checks

javascript

// Health check endpoint

```
app.get('/health', async (req, res) => {  
  const health = {  
    status: 'ok',  
    timestamp: new Date().toISOString(),  
    uptime: process.uptime(),  
    checks: {}  
  };  
  
  // Check database  
  try {  
    await supabase.from('clients').select('count').single();  
    health.checks.database = 'ok';  
  } catch (error) {  
    health.checks.database = 'error';  
    health.status = 'degraded';  
  }  
  
  // Check Redis  
  try {  
    await redis.ping();  
    health.checks.redis = 'ok';  
  } catch (error) {  
    health.checks.redis = 'error';  
    health.status = 'degraded';  
  }  
  
  // Check OpenAI  
  try {  
    await openai.models.list();  
    health.checks.openai = 'ok';  
  } catch (error) {  
    health.checks.openai = 'error';  
    health.status = 'degraded';  
  }  
  
  const statusCode = health.status === 'ok' ? 200 : 503;  
  res.status(statusCode).json(health);  
});
```

Security & Authentication {#security}

API Key Management

javascript

// .env file (NEVER commit to git)

OPENAI_API_KEY=sk-...

SUPABASE_URL=https://...

SUPABASE_KEY=eyJ...

GOOGLE_CLIENT_ID=...

GOOGLE_CLIENT_SECRET=...

GOOGLE_REFRESH_TOKEN=...

// Load environment variables

require('dotenv').config();

// Access in code

const apiKey = process.env.OPENAI_API_KEY;

// For Make.com: Store in Connections (encrypted)

// Never hardcode in scenarios

Webhook Verification

javascript

```
// Verify webhook signatures (example: Stripe)
const crypto = require('crypto');

app.post('/webhook/stripe', (req, res) => {
  const signature = req.headers['stripe-signature'];
  const webhookSecret = process.env.STRIPE_WEBHOOK_SECRET;

  let event;

  try {
    event = stripe.webhooks.constructEvent(
      req.body,
      signature,
      webhookSecret
    );
  } catch (error) {
    console.error('Webhook signature verification failed:', error);
    return res.status(400).send('Invalid signature');
  }

  // Process event
  handleStripeEvent(event);

  res.sendStatus(200);
});

// For Make.com webhooks: Add secret token in URL
// https://hook.make.com/xxx?token=your_secret_token
// Then check: if (req.query.token !== SECRET) return 403
```

Data Encryption

javascript

```

const crypto = require('crypto');

const ENCRYPTION_KEY = process.env.ENCRYPTION_KEY; // 32 bytes
const IV_LENGTH = 16;

function encrypt(text) {
  const iv = crypto.randomBytes(IV_LENGTH);
  const cipher = crypto.createCipheriv(
    'aes-256-cbc',
    Buffer.from(ENCRYPTION_KEY),
    iv
  );

  let encrypted = cipher.update(text, 'utf8', 'hex');
  encrypted += cipher.final('hex');

  return iv.toString('hex') + ':' + encrypted;
}

function decrypt(text) {
  const parts = text.split(':');
  const iv = Buffer.from(parts[0], 'hex');
  const encrypted = parts[1];

  const decipher = crypto.createDecipheriv(
    'aes-256-cbc',
    Buffer.from(ENCRYPTION_KEY),
    iv
  );

  let decrypted = decipher.update(encrypted, 'hex', 'utf8');
  decrypted += decipher.final('utf8');

  return decrypted;
}

// Store sensitive data encrypted
await supabase.from('clients').insert({
  name: 'John Doe',
  email: 'john@example.com',
  api_key: encrypt('sensitive_api_key_here')
});

```

Rate Limiting

```
const rateLimit = require('express-rate-limit');

// General API rate limit
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per window
  message: 'Too many requests, please try again later'
});

app.use('/api/', apiLimiter);

// Stricter limit for AI endpoints
const aiLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  max: 10,
  message: 'AI request limit exceeded'
});

app.use('/api/ai/', aiLimiter);

// Per-user rate limiting
const userLimiter = async (req, res, next) => {
  const userId = req.user.id;
  const key = `rate_limit:${userId}`;

  const current = await redis.incr(key);

  if (current === 1) {
    await redis.expire(key, 60); // Reset after 1 minute
  }

  if (current > 20) {
    return res.status(429).json({ error: 'Rate limit exceeded' });
  }

  next();
};
```

Scaling Strategies {#scaling}

Horizontal Scaling

javascript

```
// Use PM2 for process management  
// ecosystem.config.js  
module.exports = {  
  apps: [{  
    name: 'automation-api',  
    script: './server.js',  
    instances: 'max', // Use all CPU cores  
    exec_mode: 'cluster',  
    env: {  
      NODE_ENV: 'production'  
    }  
  }]  
};  
  
// Start: pm2 start ecosystem.config.js  
// Monitor: pm2 monit  
// Logs: pm2 logs
```

Caching Strategy

javascript

```
const Redis = require('ioredis');
const redis = new Redis();

// Cache function results
async function getCachedOrFetch(key, fetchFn, ttl = 3600) {
  // Try cache first
  const cached = await redis.get(key);
  if (cached) {
    return JSON.parse(cached);
  }

  // Fetch if not cached
  const data = await fetchFn();

  // Store in cache
  await redis.setex(key, ttl, JSON.stringify(data));

  return data;
}

// Usage
const leadData = await getCachedOrFetch(
  `lead:${leadId}`,
  async () => {
    const { data } = await supabase
      .from('leads')
      .select('*')
      .eq('id', leadId)
      .single();
    return data;
  },
  300 // 5 minutes
);

// Invalidate cache on update
async function updateLead(leadId, updates) {
  await supabase
    .from('leads')
    .update(updates)
    .eq('id', leadId);

  // Invalidate cache
  await redis.del(`lead:${leadId}`);
}
```

Database Optimization

sql

-- Add indexes for frequently queried columns

CREATE INDEX idx_leads_status **ON** leads(status);

CREATE INDEX idx_leads_score **ON** leads(score **DESC**);

CREATE INDEX idx_leads_created_at **ON** leads(created_at **DESC**);

CREATE INDEX idx_automation_runs_automation_id **ON** automation_runs(automation_id);

CREATE INDEX idx_automation_runs_executed_at **ON** automation_runs(executed_at **DESC**);

-- Composite indexes for common query patterns

CREATE INDEX idx_leads_status_score **ON** leads(status, score **DESC**);

CREATE INDEX idx_automation_runs_status_date **ON** automation_runs(status, executed_at **DESC**);

-- Partial indexes for specific conditions

CREATE INDEX idx_hot_leads **ON** leads(score) **WHERE** score **>=** 70;

CREATE INDEX idx_failed_runs **ON** automation_runs(executed_at) **WHERE** status = 'failed';

-- Query optimization example

-- Before (slow):

SELECT * FROM leads **WHERE** status = 'new' **ORDER BY** score **DESC LIMIT** 10;

-- After (fast with index):

-- Uses idx_leads_status_score composite index

Queue Optimization

javascript

```
// Prioritize jobs based on importance
await emailQueue.add(
  'send-email',
  { to, subject, body },
  {
    priority: email.urgent ? 1 : 10, // Lower number = higher priority
    attempts: 3,
    backoff: {
      type: 'exponential',
      delay: 2000
    }
  }
);

// Batch processing for efficiency
const BATCH_SIZE = 100;

new Worker('email-batch', async (job) => {
  const recipients = job.data.recipients;

  // Process in batches
  for (let i = 0; i < recipients.length; i += BATCH_SIZE) {
    const batch = recipients.slice(i, i + BATCH_SIZE);

    await Promise.all(
      batch.map(recipient => sendEmail(recipient))
    );

    // Update progress
    await job.updateProgress((i / recipients.length) * 100);

    // Small delay between batches to avoid rate limits
    await new Promise(resolve => setTimeout(resolve, 1000));
  }
});
```

Code Library {#code-library}

Utility Functions

javascript

```
// utils/date.js
```

```
exports.formatDate = (date, format = 'YYYY-MM-DD') => {  
  const d = new Date(date);  
  const year = d.getFullYear();  
  const month = String(d.getMonth() + 1).padStart(2, '0');  
  const day = String(d.getDate()).padStart(2, '0');  
  
  return format  
    .replace('YYYY', year)  
    .replace('MM', month)  
    .replace('DD', day);  
};
```

```
exports.addDays = (date, days) => {  
  const result = new Date(date);  
  result.setDate(result.getDate() + days);  
  return result;  
};
```

```
exports.isBusinessDay = (date) => {  
  const day = new Date(date).getDay();  
  return day !== 0 && day !== 6; // Not Sunday or Saturday  
};
```

```
// utils/validation.js
```

```
exports.isValidEmail = (email) => {  
  const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
  return regex.test(email);  
};
```

```
exports.isValidPhone = (phone) => {  
  // Indian phone number  
  const regex = /^[6-9]\d{9}$/;  
  return regex.test(phone.replace(/\D/g, ""));  
};
```

```
exports.sanitizeInput = (input) => {  
  return input  
    .trim()  
    .replace(/[\<>]/g, "") // Remove potential HTML tags  
    .substring(0, 1000); // Limit length  
};
```

```
// utils/string.js
```

```
exports.truncate = (str, length = 100) => {  
  if (str.length <= length) return str;
```

```
    return str.substring(0, length) + '...';  
};  
  
exports.slugify = (str) => {  
    return str  
        .toLowerCase()  
        .replace(/^[^\w\s-]/g, "")  
        .replace(/\s+/g, '-');  
};  
  
exports.capitalize = (str) => {  
    return str.charAt(0).toUpperCase() + str.slice(1).toLowerCase();  
};
```

Email Templates

javascript

```
// templates/emails.js
```

```
exports.leadConfirmation = ({ name, company }) => ({
  subject: `Thanks for your interest, ${name}!`,
  html: `
    <h2>Hi ${name},</h2>
    <p>Thanks for reaching out! We received your inquiry about automating ${company}'s workflows.</p>
    <p>Here's what happens next:</p>
    <ol>
      <li>We'll review your information (takes ~2 hours)</li>
      <li>A team member will reach out within 24 hours</li>
      <li>We'll schedule a quick demo to show you what's possible</li>
    </ol>
    <p>In the meantime, check out our case studies: <a href="https://yoursite.com/cases">View Case Studies</a></p>
    <p>Questions? Just reply to this email.</p>
    <p>Best,<br>Your Team</p>
  `
});
```

```
exports.hotLeadAlert = ({ lead }) => ({
  subject: `🔥 HOT LEAD: ${lead.name} from ${lead.company}`,
  html: `
    <h2>New Hot Lead!</h2>
    <table>
      <tr><td><strong>Name:</strong></td><td>${lead.name}</td></tr>
      <tr><td><strong>Company:</strong></td><td>${lead.company}</td></tr>
      <tr><td><strong>Email:</strong></td><td>${lead.email}</td></tr>
      <tr><td><strong>Phone:</strong></td><td>${lead.phone}</td></tr>
      <tr><td><strong>Budget:</strong></td><td>₹${lead.budget}</td></tr>
      <tr><td><strong>Score:</strong></td><td>${lead.score}/100</td></tr>
      <tr><td><strong>Timeline:</strong></td><td>${lead.timeline}</td></tr>
    </table>
    <p><strong>Needs:</strong> ${lead.needs}</p>
    <p><a href="https://crm.com/leads/${lead.id}">View in CRM</a></p>
  `
});
```

```
exports.dailyReport = ({ date, metrics, insights }) => ({
  subject: `Daily Report - ${date}`,
  html: `
    <h1>Daily Performance Report</h1>
    <p><strong>Date:</strong> ${date}</p>

    <h2>Automations</h2>
    <ul>
      <li>Total Runs: ${metrics.automations.totalRuns}</li>
      <li>Success Rate: ${metrics.automations.successRate}%</li>
    </ul>
  `
});
```

```
    <li>Avg Duration: ${metrics.automations.avgDuration}ms</li>
  </ul>

  <h2>Leads</h2>
  <ul>
    <li>New Leads: ${metrics.leads.newLeads}</li>
    <li>Hot Leads: ${metrics.leads.hotLeads}</li>
    <li>Avg Score: ${metrics.leads.avgScore}</li>
  </ul>

  <h2>AI Insights</h2>
  <div style="background: #f0f0f0; padding: 15px; border-left: 4px solid #0066cc;">
    ${insights}
  </div>
  ,
  });
```

Prompt Templates

```
javascript
```

// templates/prompts.js

```
exports.classifyEmail = ({ subject, body }) => `
```

You are an email classification system for a B2B automation company.

Email Subject: \${subject}

Email Body: \${body}

Classify this email into EXACTLY ONE category and return ONLY the category name:

- SALES: Customer wants to buy, inquire about products/pricing, or schedule demo
- SUPPORT: Customer needs help with existing service, reporting issues, or asking how-to questions
- BILLING: Questions about invoices, payments, refunds, or pricing changes
- PARTNERSHIP: Proposing collaboration, integration, or business partnership
- SPAM: Unsolicited offers, obvious spam, or completely irrelevant

Category: `;

```
exports.scoreLead = ({ name, email, company, budget, timeline, needs, role }) => `
```

Score this B2B lead from 0-100 based on fit and intent.

Lead Details:

- Name: \${name}
- Email: \${email}
- Company: \${company || 'Not provided'}
- Role: \${role || 'Not provided'}
- Budget: \${budget || 'Not provided'}
- Timeline: \${timeline || 'Not provided'}
- Needs: \${needs || 'Not provided'}

Scoring Criteria:

1. Budget (0-30 points):

- >₹1L: 30 points
- ₹50k-1L: 20 points
- ₹20k-50k: 10 points
- <₹20k: 5 points
- Unknown: 0 points

2. Timeline (0-25 points):

- ASAP/This week: 25 points
- This month: 20 points
- Next 3 months: 10 points
- Exploring/6+ months: 5 points

3. Decision Maker (0-25 points):

- CEO/Founder/Owner: 25 points
- Director/VP: 20 points

- Manager: 15 points
- Other: 5 points

4. Clarity of Needs (0-20 points):

- Specific, detailed: 20 points
- General idea: 10 points
- Vague: 5 points

Return ONLY a JSON object:

```
{  
  "score": number,  
  "classification": "HOT" | "WARM" | "COLD",  
  "reasoning": "One sentence explanation",  
  "next_action": "Specific recommendation"  
};
```

```
exports.generateResponse = ({ context, customerEmail, tone = 'professional' }) => `
```

Generate a customer email response.

Context: \${context}

Tone: \${tone}

Customer Email: \${customerEmail}

Guidelines:

- Keep under 150 words
- Be helpful and specific
- Include clear next steps
- Use friendly \${tone} tone
- If you can't help, explain why politely

Response: `;

Performance Benchmarks

Expected Response Times

Email Classification (OpenAI): 1-3 seconds

Lead Scoring (AI): 2-4 seconds

Database Query (simple): 10-50ms

Database Query (complex): 100-500ms

Webhook Processing: <500ms

Full Automation Workflow: 5-30 seconds

Targets:

- 95% of requests under 5 seconds
- 99% of requests under 10 seconds
- Zero data loss
- 99.5% uptime

Cost Estimates (Monthly)

Phase 1 (1-3 months, 0-5 clients):

- Make.com: Free - ₹800
- OpenAI API: ₹1,500 - ₹3,000
- Domain/Hosting: ₹500
- Total: ₹2,000 - ₹4,000

Phase 2 (4-6 months, 5-10 clients):

- Make.com Pro: ₹1,500
- OpenAI API: ₹5,000
- Airtable: ₹1,500
- Twilio: ₹2,000
- Supabase: Free - ₹2,000
- Total: ₹10,000 - ₹12,000

Phase 3 (7-12 months, 10-20 clients):

- Make.com Enterprise: ₹8,000
- OpenAI API: ₹15,000
- Infrastructure: ₹10,000
- Third-party APIs: ₹5,000
- Total: ₹40,000 - ₹50,000

Troubleshooting Guide

Common Issues

Issue: Make.com scenario fails silently

Solution:

1. Check operation count (might be at limit)
2. Look at execution history for errors
3. Add error handler modules
4. Enable email notifications for failures

Issue: OpenAI API returns errors

Common causes:

- Rate limit exceeded (429 error)
- Invalid API key (401 error)
- Malformed request (400 error)
- Context length exceeded (tokens > limit)

Solutions:

- Implement rate limiting
- Add retry logic with backoff
- Validate requests before sending
- Truncate long inputs

Issue: Webhook not triggering

Checklist:

- Is the webhook URL correct?
- Is the service actually sending webhooks?
- Check firewall/security settings
- Test with webhook.site first
- Verify authentication (if required)

Issue: Database queries slow

Solutions:

- Add indexes on queried columns
- Use connection pooling
- Implement caching layer
- Optimize query (use EXPLAIN ANALYZE)
- Consider read replicas

Next Steps

Week 1 Tasks

1. Set up Make.com account
2. Build email auto-responder demo
3. Get OpenAI API key
4. Test basic classification

Week 2 Tasks

1. Create Google Sheets database
2. Build lead scoring automation
3. Test end-to-end workflow
4. Document everything

Week 3-4 Tasks

1. Find first client
2. Customize automations for them
3. Deploy and monitor
4. Collect feedback

Month 2+ Tasks

1. Build reusable templates
 2. Explore advanced integrations
 3. Consider custom development
 4. Scale infrastructure
-

Resources

Documentation

- Make.com Docs: <https://www.make.com/en/help>
- OpenAI API: <https://platform.openai.com/docs>
- Supabase: <https://supabase.com/docs>
- BullMQ: <https://docs.bullmq.io>

Learning

- Make.com Academy (free courses)
- FreeCodeCamp Node.js tutorials
- Web Dev Simplified (YouTube)
- Traversy Media (YouTube)

Communities

- [Make.com Community Forum](#)
 - [r/nocode, r/automation \(Reddit\)](#)
 - [Indie Hackers](#)
 - [Dev.to](#)
-

Document Version: 2.0

Last Updated: January 2026

Maintained By: AI Ops Automation Project